Citi Bike rebalancing: background

Prefer to listen? 01-background.mp3 reads this document aloud (about 14 minutes).

Everything you need before the figures make sense: how Citi Bike works, what the data is, how we turn it into something we can analyse, and the few ideas the whole project rests on. The figure-by-figure guide is `02-figures-and-model.md`.

1. How Citi Bike works

Citi Bike is New York's bike share, run by Lyft for the city. There are about 2,500 stations across Manhattan, Brooklyn, Queens, the Bronx, Jersey City and Hoboken, and about 39,000 bikes. A station is a row of docks, typically 23 (the biggest has 123). You take a bike from a dock, ride, and return it to any free dock at any station. Most trips are short: the median is 9 minutes, and only 2% end where they started.

Two kinds of rider matter:

- **Members** pay yearly and take 82% of all trips. They are mostly commuters, so they ride at fixed times on fixed routes.
- **Casual riders** buy single rides or day passes. They ride more at weekends and midday, spread out over the day.

71% of trips are on e-bikes. For this project a bike is a bike: an e-bike takes a dock like any other.

2. The business problem

Every workday morning, members ride from where they live (Brooklyn, the East Village, the Upper West Side) to where they work (Midtown, Lower Manhattan). In the evening they ride back. Nobody rides the other way in the same numbers, so bikes pile up on one side:

- In the morning, **home stations empty out**. The next person finds no bike.
- At the same time, **office stations fill up**. The next person arriving finds no free dock and has to ride on to another station.

Both are a rider turned away. Citi Bike fixes this by moving bikes back against the flow: vans and box trucks for bulk moves, bike trains (e-bikes towing a trailer of 12 to 16 bikes), valets who take extra bikes at the busiest stations, and Bike Angels, which pays members in points to ride from full stations to empty ones. By its own monthly reports it moves about 2,500 bikes a day.

Moving bikes costs staff, fuel and trucks. A bike moved to a station that didn't need it costs just as much and prevents nothing. So the question is:

> **Each night, which stations should Citi Bike restock or clear, and by how many bikes, to turn away the fewest riders for each bike it moves? And is a second visit during the day worth it?**

Citi Bike publishes how many bikes it moves each month, but not which stations, to what level, or how many riders still find a station empty. So we can't score Citi Bike's own plan. We compare our plans against two simple benchmarks instead: **never moving a bike**, and **resetting every station to half full each night**. Then we check that our workload is realistic against Citi Bike's reported numbers.

3. The data

Five public sources, all free, all downloaded by `lib/fetching.py`.

Source	What we use it for	Size
Citi Bike trip records	Every trip: where and when it started and ended	92 million trips, Oct 2024 to Sep 2026, 24 monthly files
Citi Bike live station feed (GBFS), pulled 22 Oct 2026	How many docks each station has	2,551 stations, 81 of them without a dock count

Source	What we use it for	Size
Citi Bike monthly operating reports to NYC DOT	How many bikes Citi Bike moved each month, for the reality check	20 PDFs, Jan 2025 to Sep 2026
Open-Meteo hourly weather, Central Park	Temperature, rain, snow and wind, for the demand model	17,520 hours
NYC Open Data and New Jersey GIS	Borough, neighbourhood and town outlines, for the maps	262 neighbourhoods, 12 NJ towns

The trip files, column by column

One row per trip, 13 columns:

Column	Example	Used for
`ride_id`	2DBFC5973700F55D	Nothing (a unique id)
`rideable_type`	electric_bike, classic_bike	Counting e-bike trips
`started_at`, `ended_at`	2024-10-10 11:03:32	When the bike left and arrived, to the second
`start_station_name`, `end_station_name`	Broadway & W 25 St	Station names for the station table
`start_station_id`, `end_station_id`	7293.10	**The key to everything**: which station lost a bike, which gained one
`start_lat`, `start_lng`, `end_lat`, `end_lng`	40.697, -73.923	Station positions for the maps
`member_casual`	member, casual	Splitting commuters from casual riders (figure 1)

What the trip files **do not** have, and what that means for the project:

1. **How many bikes are in each dock.** Never published. We rebuild it (section 4).
2. **Citi Bike's own truck moves.** A bike a truck moved never shows up as a trip.
3. **Riders who were turned away.** Someone who found an empty station and walked off never made a trip, so they are invisible. Our shortage counts are therefore lower bounds: the real numbers are at least this high.

The weather file

Hourly: `time`, `temperature_2m (°C)`, `precipitation (mm)`, `snowfall (cm)`, `wind_speed_10m (km/h)`. We turn it into one row per day: average temperature 07:00 to 21:00 (riding hours), number of wet hours (at least 0.5 mm), snowfall over the last three days (snow stays on the bike lanes), and average wind.

4. How the raw data becomes something we can analyse

This is `prepare_data.py` and `station_days.py`. It runs one month at a time, so memory stays around one month of trips (2.8 GB at peak instead of 15 to 20 GB).

Step 1: cleaning

Four problems in the raw files, all fixed in `prepare_data.py`:

1. **Station ids lose a trailing zero in some months.** 7293.10 is written as 7293.1 and stops matching its station. Fix: any id with one decimal gets the zero back. Temporary endings like `_old` are dropped too.
2. **Citi Bike's own sites look like stations**: depots, warehouses, a charging hub, labs, three Los Angeles demo units. Fix: only ids shaped like a public station (`1234.56`, or `JC123` and `HB123` in New Jersey) are kept; the rest become null, so those trips don't count at any station.

3. **One monthly file repeats 512 trips** that belong to the next month. Each file holds the trips that ended in that month, so a trip is kept only from the file of the month it ended in.
4. **23 February 2026 has no trips at all**: the system shut for 22 cm of snow. The demand model leaves that day out, since weather can't explain a closure.

After cleaning, 99.9% of trips start at a public station and 99.7% end at one.

Step 2: counting bikes in and out per station, every 5 minutes

For each month, two group-bys on the trip table:

- **Starts**: group trips by start station and `started_at` rounded down to 5 minutes, and count. Also count how many were members and how many were e-bikes.
- **Ends**: group trips by end station and `ended_at` rounded down to 5 minutes, and count.

Then a full outer join of the two on (station, slot): a slot can have starts and no ends, or the other way round, and a missing side becomes 0. The result is `data/processed/bins/YYYYMM.parquet`, one row per station per 5-minute slot with any activity. That's about 4.7 million rows a month:

station_id	slot	starts	member_starts	ebike_starts	ends
1964.01	2026-05-01 16:15	2	0	2	0

Why 5 minutes: fine enough to see a station fill up during the rush, coarse enough to keep the tables small. Hourly would hide when exactly a station runs out.

Step 3: the station table

One row per station: id, name, latitude, longitude, docks. The trip files repeat each station's name and position on every trip, so the name is the one most trips used and the position is the median over its trips. Docks come from the live feed, joined on the station id; where a station was renumbered, the join falls back to the name. Stations without a dock count (81, holding 4.1% of trip ends) are left out of the simulation, which leaves 2,470.

Step 4: the trip table for the simulation

`data/processed/trips/YYYYMM.parquet`: every trip from a public station, with start station, end station, start slot and end slot. The simulation needs this because of the rule in section 6: a rider turned away at the start never arrives at the end.

Step 5: the daily swing per station

`station_days.py` turns the 5-minute counts into one row per station per day (1.6 million rows). The day runs from **04:00 to 04:00**, because 04:00 is the quietest hour: almost nobody rides, so that's when trucks reset stations and when we measure from.

For each station and day it adds up arrivals minus departures, slot by slot. That running total is the **drift**: how many bikes the station has gained since 04:00. It stores the lowest and highest drift of the day (and of the rest of the day from 10:00, 12:00 and 14:00, for the midday visit).

5. The three ideas the project rests on

Idea 1: we can rebuild every station's stock from trips alone

A station's bike count only changes when a bike leaves or arrives. So if you know how many bikes it had at 04:00 and you add up arrivals minus departures, you know how many it has at every moment of the day. We never see the 04:00 count, but we don't need it: that's exactly what a schedule decides. Set a starting level, replay the day, and you know whether the station ran empty or full.

This turns public trip records into a simulator: replay a year of real riders under any truck plan and count the riders it would have turned away. No one has to drive a truck to test a plan.

Idea 2: need and room

From the drift, two numbers decide whether a starting level works on a given day:

- **Need** = how far the drift falls below zero. Start with fewer bikes than that and the station runs empty at some point.
- **Room** = docks minus how far the drift rises. Start with more bikes than that and the station fills up.

Any start between need and room gets the station through the whole day. Example: Broadway & W 25 St (109 docks) on 12 May 2026 gained 105 bikes over the morning and never fell below its 04:00 level. Need 0, room 109 − 105 = 4. Any start from 0 to 4 bikes works; half full (54) fills up by 08:45 and turns away 50 riders.

Idea 3: the target band, from each station's own last 10 days

We don't know tomorrow's need and room. But stations repeat themselves (figure 4: correlation 0.998 week to week), so we look at the station's last 10 days of the same type (workdays with workdays, weekends with weekends). For each possible starting level x, add up the riders it would have turned away over those 10 days: need − x on days where x was too low, x − room on days where it was too high. The range of x where that total is lowest is the station's **target band**. It works out to the median of the 20 need and room numbers.

A station already inside its band is left alone. One outside it is moved to the nearest edge. That's the core rule. The rest is about spending a limited number of bike moves well (a night budget, taking the moves that save the most first) and adding a 14:00 visit before the evening rush.

6. How we test it fairly

- **Two years, split by time.** The first year (Oct 2024 to Sep 2025) is for exploring and choosing every setting. The second year (Oct 2025 to Sep 2026) is used once, at the end, to score the final plans. Nothing about the plan was tuned on the year that scores it.
- **Every plan faces exactly the same riders.** So even though the shortage counts are lower bounds, the comparison between plans is fair.
- **A turned-away rider never arrives.** The real trips already include Citi Bike's own truck moves: some riders could only ride because a truck had brought a bike. So if a plan leaves a station empty, the riders it turns away must also be struck from their destinations' arrivals, since those bikes never get there. Without this rule, never moving a bike looks better than resetting to half full, which is nonsense. With it, never moving a bike turns away 18,189 riders a day.
- **A reality check against Citi Bike.** Our recommended plan moves 2,572 bikes a day in the months Citi Bike reports on, against Citi Bike's 2,514, and the two rise and fall together month to month (correlation 0.92).

7. The answer, in one paragraph

Resetting every station to half full each night moves 5,900 bikes a day and still turns away 4,700 riders. Our plan gives each station a target band from its last 10 comparable days and touches only stations outside it. It moves the bikes that prevent the most shortages first, up to 3,000 bikes loaded or unloaded a night, and sends vans to the 100 neediest stations at 14:00. On the test year it turns away 1,779 riders a day (1.5 per 100 trips) and moves 2,549 bikes, about what Citi Bike already moves. That's **62% fewer riders turned away with 57% fewer bikes moved**. A quarter of what's left sits at seven stations whose daily swing is bigger than their docks. No truck plan fixes those; they need more docks or a valet.

8. Words you'll hear

Term	Meaning
Slot	A 5-minute window. A day has 288
Service day	04:00 to 04:00 the next morning
Net flow	Bikes arriving minus bikes leaving, over some period

Term	Meaning
Drift	Net flow added up from 04:00: bikes gained since the start of the day
Need	How far the drift falls below zero: the fewest bikes to start with and never run empty
Room	Docks minus the highest drift: the most bikes to start with and never fill up
Target band	The range of starting levels that would have turned away the fewest riders over a station's last 10 comparable days
Shortage / rider turned away	Someone who found no bike, or no free dock
Bikes moved	Bikes picked up at one station and dropped at another (the larger of pick-ups and drop-offs)
Budget	The most bikes loaded or unloaded in a night
Dev year / test year	Oct 2024 to Sep 2025, for choosing; Oct 2025 to Sep 2026, for scoring once
Oracle / band known in advance	The best start for the day that actually happened: the best any overnight plan could do (869 a day)